MODEL EVALUATION AND IMPROVEMENT:

HYPERPARAMETER TUNING WITH GRID SEARCH AND CROSS-VALIDATION

Experiment No.	8	**Date**	10-09-26

AIM

To demonstrate key techniques for model evaluation and improvement:

1. **Hyperparameter Tuning with Grid Search:** Systematically searching for the optimal combination of hyperparameters for a machine learning model.
2. **Cross-Validation Techniques:** Implementing k-fold cross-validation to get a more robust estimate of model performance and to prevent overfitting to a specific train-test split.

ALGORITHM

1. Hyperparameter Tuning with Grid Search

Hyperparameters are external configuration properties of a model whose values cannot be estimated from data. Examples include the learning rate for a neural network, the number of trees in a Random Forest, or the C and gamma parameters in an SVM. Tuning these parameters is crucial for optimal model performance.

Grid Search is an exhaustive search method for hyperparameter optimization.

Steps:

1. Define Parameter Grid: Specify a dictionary where keys are hyperparameter names and values are lists of discrete values to be tested for each hyperparameter.
2. Instantiate Model: Choose a machine learning model.
3. Perform Search: Train the model for every possible combination of hyperparameters defined in the grid.
4. Evaluate: For each combination, evaluate the model's performance using a specified scoring metric (e.g., accuracy, F1-score) and often in conjunction with cross-validation.
5. Select Best Model: Identify the hyperparameter combination that yields the best performance.

2. Cross-Validation Techniques

Cross-validation is a resampling procedure used to evaluate machine learning models on a limited data sample. The goal is to estimate how accurately a predictive model will perform in practice. It is especially useful for reducing overfitting and providing a more reliable estimate of generalization performance compared to a single train-test split.

k-Fold Cross-Validation - Steps:

1. Divide Data: The entire dataset is randomly partitioned into k equally sized subsamples (or “folds”).
2. Iterate k Times: In each iteration, one fold is used as the validation (or test) set, and the remaining k-1 folds are used as the training set. The model is trained on the training set and evaluated on the validation set.

3 Aggregate Results: The performance metric (e.g., accuracy) from each of the k iterations is collected.

4 Compute Mean and Standard Deviation: The mean and standard deviation of these k performance scores are calculated to provide a more robust estimate of the model's performance and its variability.

CODE

Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, KFold, cross_val_score, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
```

Part 1: Hyperparameter Tuning with Grid Search

Step 1 – Load the Iris Dataset

```
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names

print(f"Dataset Features (X) shape: {X.shape}")
print(f"Dataset Labels (y) shape: {y.shape}")
print(f"Feature Names: {feature_names}")
print(f"Target Names: {target_names}")
```

```
Dataset Features (X) shape: (150, 4)
Dataset Labels (y) shape: (150,)
Feature Names: ['sepal length (cm)', 'sepal width (cm)',
'petal length (cm)', 'petal width (cm)']
Target Names: ['setosa' 'versicolor' 'virginica']
```

Step 2 – Train/Test Split and Feature Scaling

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.3, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
Training set size: 105 samples
Test set size: 45 samples
Features standardized.
```

Step 3 – Define the Hyperparameter Grid

```
param_grid = {
'C': [0.1, 1, 10, 100], # Regularization parameter
'gamma': [1, 0.1, 0.01, 0.001], # Kernel coefficient
'kernel': ['rbf', 'linear'] # Kernel type
}
```

Step 4 – Perform Grid Search with 5-Fold Cross-Validation

```
grid_search = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy',
verbose=1, n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)

print(f"Best hyperparameters found: {grid_search.best_params_}")
print(f"Best cross-validation accuracy: {grid_search.best_score_:.4f}")
```

```
Fitting 5 folds for each of 32 candidates, totalling 160 fits
Best hyperparameters found: {'C': 1, 'gamma': 0.1, 'kernel': 'rbf'}
Best cross-validation accuracy: 0.9810
```

Step 5 – Evaluate the Best Model on the Test Set

```
best_model = grid_search.best_estimator_
y_pred_tuned = best_model.predict(X_test_scaled)
test_accuracy_tuned = accuracy_score(y_test, y_pred_tuned)

print(f"Test set accuracy with tuned model: {test_accuracy_tuned:.4f}")
print(classification_report(y_test, y_pred_tuned, target_names=target_names))
```

```
Test set accuracy with tuned model: 0.9111
```

Classification Report – Tuned Model

Class	Precision	Recall	F1-Score	Support
setosa	1.00	1.00	1.00	15
versicolor	0.82	0.93	0.88	15
virginica	0.92	0.80	0.86	15
accuracy			0.91	45
macro avg	0.92	0.91	0.91	45
weighted avg	0.92	0.91	0.91	45

Confusion Matrix Visualization

```
cm_tuned = confusion_matrix(y_test, y_pred_tuned)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_tuned, annot=True, fmt='d', cmap='Blues',
xticklabels=target_names, yticklabels=target_names)
plt.title('Confusion Matrix (Tuned SVM)')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

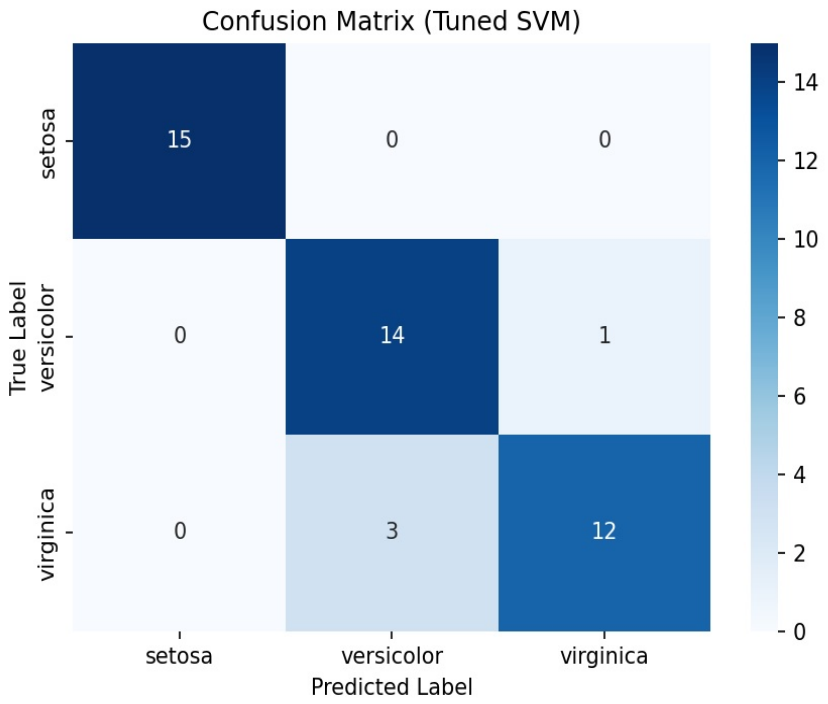

Fig. 1 – Confusion Matrix (Tuned SVM)

Top 5 Grid Search Results

```
results_df = pd.DataFrame(grid_search.cv_results_)
print(results_df[['param_C', 'param_gamma', 'param_kernel',
'mean_test_score', 'rank_test_score']]
.sort_values(by='rank_test_score').head())
```

C	Gamma	Kernel	Mean Test Score	Rank
1.0	0.1	rbf	0.980952	1
100.0	0.1	linear	0.980952	1
100.0	0.001	linear	0.980952	1
100.0	0.01	linear	0.980952	1
100.0	1.0	linear	0.980952	1

Part 2: Cross-Validation Techniques (k-Fold)

```
model_cv = SVC(random_state=42)

k_folds = 5
kf = KFold(n_splits=k_folds, shuffle=True, random_state=42)

cv_scores = cross_val_score(model_cv, X_train_scaled, y_train, cv=kf, scoring='accuracy')

print(f"Cross-validation scores for each fold: {cv_scores}")
print(f"Mean cross-validation accuracy: {np.mean(cv_scores):.4f}")
print(f"Standard deviation of cross-validation accuracy: {np.std(cv_scores):.4f}")
```

```
Performing 5-fold cross-validation...

Cross-validation scores for each fold: [1. 0.9524 0.9524 0.9524 1.]
Mean cross-validation accuracy: 0.9714
Standard deviation of cross-validation accuracy: 0.0233
```

Cross-Validation Score Visualization

```
plt.figure(figsize=(8, 5))
plt.bar(range(1, k_folds + 1), cv_scores, color='skyblue')
plt.axhline(y=np.mean(cv_scores), color='r', linestyle='--',
label=f'Mean Accuracy ({np.mean(cv_scores):.4f})')
plt.title(f'{k_folds}-Fold Cross-Validation Accuracy Scores')
plt.xlabel('Fold Number')
plt.ylabel('Accuracy')
plt.ylim(0.8, 1.0)
plt.legend()
plt.grid(axis='y', linestyle='--')
plt.show()
```

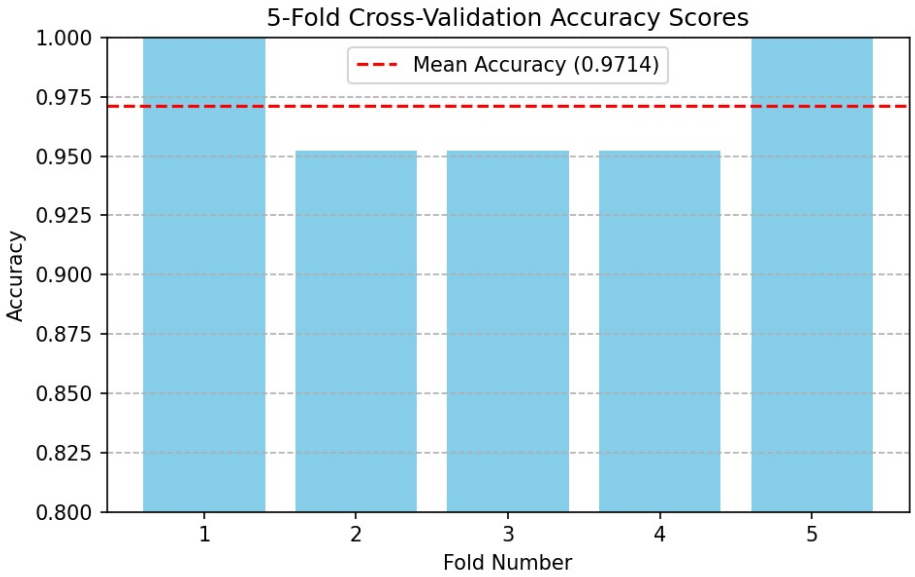

Fig. 2 – 5-Fold Cross-Validation Accuracy Scores

Why is Cross-Validation Important?

```
print("1. More Reliable Performance Estimate: Reduces bias from a single train-test split.")
print("2. Better Generalization: Helps ensure the model performs well on unseen data.")
print("3. Efficient Data Usage: All data points are used for both training and validation.")
print("4. Detects Overfitting/Underfitting: Variability in scores can indicate
instability.")
```

1 **More Reliable Performance Estimate:** Reduces bias from a single train-test split.

2 **Better Generalization:** Helps ensure the model performs well on unseen data.

3 **Efficient Data Usage:** All data points are used for both training and validation across different folds.

4 **Detects Overfitting/Underfitting:** Variability in scores can indicate instability.

RESULT

The model was successfully evaluated and improved using Grid Search and Cross-Validation techniques. Grid Search identified the best combination of hyperparameters (C = 1, gamma = 0.1, kernel = 'rbf'), achieving a best cross-validation accuracy of **0.9810**.

Cross-Validation ensured reliable performance estimation, giving a mean accuracy of **0.9714** with a standard deviation of **0.0233** across 5 folds.

The optimized model achieved a test set accuracy of **0.9111**, confirming that systematic tuning and validation significantly enhance model performance and generalization.